

# Portal Corporativo de Monitoreo de Flotas

Informe de Ejecución del Proyecto: de PLAN.md a la entrega final con Spec-Driven Development (OpenSpec)

Clean Architecture

Event-Driven

Agentic AI

Offline-First Mobile

Spec-Driven Development

**Candidato:** Julian Saldarriaga

**Repositorio:** [github.com/juliansaldar/portal-corporativo-flotas](https://github.com/juliansaldar/portal-corporativo-flotas) (público)

**Fecha del informe:** 24 de julio de 2026

**Propósito:** documento de estudio y sustentación del proceso de ejecución, preparado para exposición al cliente evaluador.

## • Contenido

---

- 1 Resumen ejecutivo
- 2 Contexto: qué exigía la prueba técnica
- 3 Metodología: planificación previa y Spec-Driven Development
- 4 Stack tecnológico y justificación de cada decisión
- 5 Arquitectura del sistema
- 6 Arquitectura de directorios del repositorio
- 7 Ejecución por bloques: los 5 changes de OpenSpec
- 8 Iteraciones de refinamiento posteriores al MVP
- 9 Auditoría de IA: criterio propio sobre código generado
- 10 Pruebas y verificación funcional
- 11 Estado de entregables
- 12 Conclusión

## 1. Resumen ejecutivo

---

Este informe documenta, de principio a fin, el proceso con el que se ejecutó el take-home assessment "Portal Corporativo de Monitoreo de Flotas": desde la sesión de planificación registrada en `PLAN.md`, pasando por la disciplina de **Spec-Driven Development con OpenSpec** (11 changes completos, cada uno con proposal → specs → design → tasks → implementación → archive), hasta la verificación funcional final del sistema completo.

El resultado es un MVP funcional que cubre los cinco bloques exigidos por el enunciado — ingesta de telemetría orientada a eventos, agente de IA conversacional, portal web reactivo, app móvil offline-first y pruebas de caos/carga con laC de referencia — más seis iteraciones adicionales de refinamiento (experiencia visual, mapa real en la app móvil, feed de telemetría en vivo) que llevaron el proyecto de "funcionalmente correcto" a "demostrable como producto".

### ENTREGABLES

Los cuatro entregables exigidos en la sección 7 del enunciado están completos y verificados: repositorio Git, README con instrucciones e laC, Auditoría de IA con 7 casos documentados, y el video de sustentación.

Este documento resume el **cómo** se llegó ahí, no solo el **qué** se construyó: es el objetivo explícito del enunciado ("evaluamos el cómo, no el qué") y por eso cada sección siguiente conecta una decisión técnica con la razón de negocio o arquitectónica que la motivó.

## 2. Contexto: qué exigía la prueba técnica

---

El enunciado ( `Prueba_Tecnica_Senior_Fullstack_Nivel_Ejecutivo.pdf` ) plantea explícitamente que se evalúa la capacidad de **orquestrar IA agéntica como arquitecto senior**, no de picar código a mano, y fija cinco bloques de trabajo:

| Bloque                                     | Exigencia del enunciado                                                                                            |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>A — Ingesta orientada a eventos</b>     | Bus de eventos asíncrono, persistencia especializada para series de tiempo, Circuit Breakers entre microservicios. |
| <b>B — Desarrollo agéntico</b>             | Agente integrado al backend que responda en lenguaje natural sobre el estado de la flota.                          |
| <b>C — Portal corporativo</b>              | Dashboard SPA con datos en tiempo real (WebSockets/SSE): mapa, alertas, chat con IA.                               |
| <b>D — Ecosistema móvil</b>                | App offline-first para conductores, persistencia local, sync en bloque, CI/CD móvil.                               |
| <b>E — Infraestructura, caos y testing</b> | Simulación de carga con duplicados/errores inyectados, IaC de referencia, Docker Compose local.                    |

Además, exige transparencia sobre el uso de IA: una Auditoría de IA con al menos dos casos donde el IDE agéntico propuso algo deficiente y se corrigió con criterio propio, y un video de sustentación que muestre tanto la arquitectura como la forma concreta de trabajo agéntico.

### 3. Metodología: planificación previa y Spec-Driven Development

#### 3.1 — PLAN.md: la sesión de planificación antes de escribir código

Antes de tocar código, se dedicó una sesión completa a fijar en `PLAN.md` las decisiones que después serían "no renegociables" durante la implementación: stack exacto, estructura de carpetas, orden de dependencia entre los 5 bloques, convención de commits y estrategia de ramas por change. Esto es deliberado: en un ejercicio pensado para completarse en 8-12 horas apoyado en IA agéntica, la mayor fuente de riesgo no es la velocidad de escritura de código sino la deriva de alcance — que el agente reintroduzca frameworks o patrones no decididos a mitad de la implementación. Fijar el plan por escrito y hacerlo la fuente de verdad convierte esa deriva en algo detectable y corregible en el momento.

#### 3.2 — AGENTS.md: el contrato de trabajo para el IDE agéntico

`AGENTS.md` es el equivalente a un `.cursorrules`: reglas explícitas que cualquier agente (Claude Code, Cursor, Copilot) debe respetar en este repositorio. Contiene el stack fijado ("no renegociable sin actualizar `PLAN.md`"), la regla arquitectónica (Clean Architecture por servicio: `domain/`, `application/`, `infrastructure/`, `interface/`), la obligación de envolver llamadas salientes en Circuit Breaker, la obligación de pasar por un change de OpenSpec antes de tocar código no trivial, la convención de commits, y la regla de seguridad de no commitar secretos. Es el artefacto que se muestra en el video como evidencia concreta de "cómo se estructuró el entorno de trabajo agéntico" — el punto que el enunciado pide dedicar 2 minutos a demostrar.

### 3.3 — OpenSpec: Spec-Driven Development como disciplina, no como burocracia

Se usó `OpenSpec` (schema `spec-driven` ) para forzar que cada bloque de trabajo pasara por un ciclo verificable antes de escribir una sola línea de código de producción:

| Etapas                        | Qué produce                                             | Por qué importa con IA agéntica                                                                       |
|-------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>proposal.md</code>      | Por qué se necesita el cambio y qué cambia a alto nivel | Obliga a articular el problema de negocio antes de que la IA proponga una solución técnica            |
| <code>specs/**/spec.md</code> | Requisitos en SHALL/MUST con escenarios WHEN/THEN       | Convierte la aceptación en algo verificable, no en "se ve bien"                                       |
| <code>design.md</code>        | Decisiones técnicas y trade-offs explícitos             | Es donde se audita y se justifica (o se rechaza) lo que sugiere la IA — la base de la Auditoría de IA |
| <code>tasks.md</code>         | Checklist granular <code>- [ ] N.M</code>               | Cada tarea es una unidad de commit — historial de git legible y auditable                             |
| <code>archive/</code>         | Cierre formal del change tras verificación funcional    | Nada se da por completo sin probarlo en vivo primero                                                  |

El resultado: **11 changes** completos y archivados en `openspec/changes/archive/` , y un historial de **137 commits** en `main` , sincronizado 1:1 con el repositorio remoto público en GitHub — sin commits directos a `main` fuera de este flujo.

### 3.4 — Entorno de desarrollo personal: por qué terminal

La disciplina de OpenSpec y el contrato de `AGENTS.md` son independientes de la herramienta — en la práctica, todo este proyecto se ejecutó desde terminal, sin un IDE gráfico tradicional de por medio:

| Herramienta                      | Rol en el flujo de trabajo                                                                                                                                                      |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Alacritty</b>                 | Emulador de terminal (aceleración por GPU) — base de todo lo demás.                                                                                                             |
| <b>Tmux</b>                      | Multiplexor de terminal — sesiones persistentes que sobreviven a un corte de SSH y paneles divididos para correr editor, logs y agentes en paralelo dentro de la misma ventana. |
| <b>Neovim + LazyVim</b>          | Editor principal, con un plugin de IA integrado para trabajar con Claude directamente sobre el buffer del código.                                                               |
| <b>lazygit</b>                   | TUI para staging, diffs y revisión de commits — el historial de 137 commits Conventional Commits de la sección 3.3 se gestionó completo desde aquí.                             |
| <b>opencode</b>                  | Agente de código en terminal, usado para alternar entre modelos Gemini y Claude según el caso.                                                                                  |
| <b>Claude Code / AntigraVity</b> | Agentes adicionales invocados directamente desde terminal para tareas puntuales del desarrollo.                                                                                 |

**POR QUÉ TERMINAL Y NO UN IDE GRÁFICO**

No es una preferencia estética: en varias ocasiones el trabajo ha requerido operar directamente sobre servidores remotos, solo por SSH y sin entorno gráfico disponible. Un flujo 100% terminal — editor, multiplexor, control de versiones y agentes de IA incluidos — es idéntico en el laptop local y en un servidor remoto, sin reconfigurar nada ni depender de extensiones de acceso remoto de un IDE gráfico.

**4. Stack tecnológico y justificación de cada decisión**

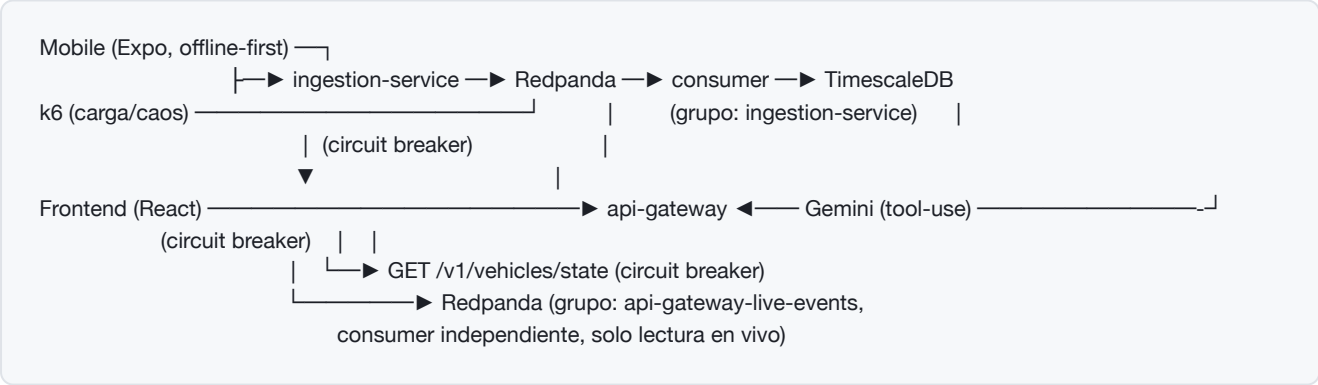
El enunciado deja libertad de stack ("nos enfocaremos en tus decisiones arquitectónicas"). Cada elección below fue una decisión consciente, documentada en `design.md` , no un default de la IA:

| Capa           | Elegido                                                                                              | Por qué                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend        | Python 3.12 + FastAPI<br>(no Node/NestJS, no Go)                                                     | Tipado con Pydantic para contratos de dominio explícitos, soporte async nativo para I/O de alta concurrencia (ingesta de miles de dispositivos), y velocidad de iteración adecuada al timebox de 8-12h sin sacrificar Clean Architecture por capas.                                                                                                                                                                                                                                      |
| Bus de eventos | Redpanda<br>(en vez de Kafka + Zookeeper)                                                            | API-compatible con Kafka (cumple el requisito literal del enunciado), pero un solo contenedor sin Zookeeper — mucho más liviano en un entorno local con recursos limitados, sin sacrificar semántica de consumer groups ni el patrón de desacople productor/consumidor que pide el bloque A.                                                                                                                                                                                             |
| Persistencia   | TimescaleDB<br>(en vez de Cassandra + Druid)                                                         | Hypertables + política de retención/compresión nativas de Postgres, pensadas exactamente para telemetría de alta frecuencia. Sumar Cassandra (para escritura distribuida) y Druid (para analítica) habría sido complejidad operativa injustificada para el volumen real de un MVP — un recorte documentado, no una omisión por desconocimiento.                                                                                                                                          |
| Agente de IA   | SDK oficial (Gemini <code>google-genai</code> , tool-use directo)<br>(sin LangChain/Semantic Kernel) | Un único tool determinístico ( <code>query_vehicle_state</code> ) no justifica la sobrecarga de un framework de orquestación de agentes. La llamada directa al SDK es más simple, más rápida y — sobre todo — más auditable: no hay una capa intermedia que oculte qué prompt o qué tool realmente se ejecutó. Migrado desde Anthropic Claude a mitad de proyecto para usar el free tier de Gemini y validar el flujo completo sin depender de créditos de pago (ver sección 9, caso 6). |
| Resiliencia    | Circuit Breaker propio<br>(~60 líneas, sin librería externa)                                         | Para un patrón closed/open/half-open reutilizado en solo 3 puntos de llamada, una dependencia externa ( <code>purgatory</code> , <code>aiobreaker</code> ) no aportaba nada que no se pudiera auditar directamente. Tenerlo en el propio repo fue justamente lo que permitió encontrar y corregir con confianza un bug real de manejo de errores (caso 4 de la Auditoría de IA).                                                                                                         |

| Capa         | Elegido                                                            | Por qué                                                                                                                                                                                                                                                                                             |
|--------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend web | React + Vite<br>(sin Next.js/SSR)                                  | Es un dashboard autenticado, no una página indexable por buscadores — SSR no aporta valor real aquí y sí añade complejidad de despliegue innecesaria. SSE (no WebSockets) para el stream de datos: es unidireccional servidor→cliente, exactamente lo que necesita un feed de telemetría y alertas. |
| Mobile       | Expo / React Native                                                | Permite probar en un dispositivo físico real (vía Expo Go) sin builds nativos ni cuentas de tienda — crítico dado el timebox.<br>Persistencia offline con <code>expo-sqlite</code> , detección de conectividad con <code>@react-native-community/netinfo</code> .                                   |
| IaC          | Terraform (ECS, Fargate, RDS, ALB, ECR)<br>documental, no aplicado | Demuestra la capacidad de diseñar el despliegue en la nube sin el riesgo/costo de una cuenta AWS real para una prueba técnica — decisión explícita, validada solo sintácticamente ( <code>terraform validate</code> ).                                                                              |

## 5. Arquitectura del sistema

El sistema se organiza como un conjunto de servicios desacoplados por un bus de eventos, con dos puntos de entrada independientes (móvil y web/carga) y dos Circuit Breakers protegiendo las llamadas salientes de `api-gateway`:



| Componente                | Rol en la arquitectura                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| backend/ingestion-service | Ingress HTTP ( /v1/telemetry , /v1/telemetry/bulk ) → Redpanda (bus de eventos, API-compatible con Kafka) → consumer idempotente → TimescaleDB. Calcula stopped_duration y pertenencia a CriticalZone por vehículo.                                                                                                                                                                                      |
| backend/api-gateway       | REST/SSE, agente de IA (Gemini, tool-use directo sobre query_vehicle_state ), llama a ingestion-service y a Gemini detrás de Circuit Breakers independientes. Además consume Redpanda directamente (segundo consumer group, independiente del de ingestion-service ) para exponer GET /v1/vehicles/{vehicle_id}/events/stream — telemetría cruda en vivo por vehículo, sin pasar por el estado agregado. |
| frontend                  | React + Vite: mapa (Leaflet), panel de alertas derivado del stream, chat con el agente, resumen de flota, roster de vehículos (seleccionable, con feed de telemetría en vivo) y guantera digital.                                                                                                                                                                                                        |
| mobile                    | Expo/React Native: captura de coordenadas offline-first (cola SQLite) con sincronización en bloque al reconectar, con pantallas de Inicio/Rastreo/Guantera/SOS por pestañas.                                                                                                                                                                                                                             |
| load-testing              | Script k6 de carga y caos (10% duplicados, 5% errores inyectados).                                                                                                                                                                                                                                                                                                                                       |
| infra                     | Terraform de referencia para AWS (documentación, no se aplica).                                                                                                                                                                                                                                                                                                                                          |

Cada bloque del enunciado (A-E) corresponde a un change de OpenSpec archivado — ver sección 7 (Ejecución por bloques) — y a la carpeta de primer nivel homónima detallada en la sección 6.

## 6. Arquitectura de directorios del repositorio

La estructura de carpetas fue fijada desde PLAN.md antes de escribir código y se mantuvo como "no renegociable" durante toda la implementación (ver sección 3.1). Cada servicio backend replica la misma separación en capas — domain/ , application/ , infrastructure/ , interface/ — exigida por AGENTS.md para cumplir Clean Architecture.



```

.
├── backend
│   ├── api-gateway      # bloque B/C — agente IA, SSE, orquesta llamadas
│   │   ├── app
│   │   │   ├── domain      # entidades y contratos (ChatModelPort, ModelTurn)
│   │   │   ├── application  # casos de uso (agent_chat, query_vehicle_state)
│   │   │   ├── infrastructure # adapters concretos (gemini_client, http clients)
│   │   │   └── interface    # routers FastAPI (REST/SSE)
│   │   └── tests
│   ├── ingestion-service # bloque A — ingesta, estado de flota
│   │   ├── app
│   │   │   ├── domain      # TelemetryEvent, CriticalZone, VehicleState
│   │   │   ├── application  # casos de uso (ingest, compute_state)
│   │   │   ├── infrastructure # Redpanda producer/consumer, TimescaleDB repo
│   │   │   └── interface    # routers FastAPI (/v1/telemetry, /internal/*)
│   │   ├── migrations
│   │   └── tests
│   ├── shared           # código compartido entre ambos servicios
│   ├── resilience       # Circuit Breaker propio (closed/open/half-open)
│   ├── messaging.py     # helpers Redpanda/Kafka
│   └── models.py        # contratos de dominio compartidos
├── frontend             # bloque C — portal SPA (React + Vite)
│   └── src
│       ├── components    # mapa, alertas, chat, roster, guantera
│       ├── hooks         # consumo de SSE
│       ├── data          # perfiles dummy de presentación
│       └── lib
├── mobile               # bloque D — app del conductor (Expo/React Native)
│   └── src
│       ├── screens       # Inicio / Rastreo / Guantera / SOS
│       ├── db            # cola offline-first (SQLite)
│       ├── api           # cliente HTTP + sync en bloque
│       └── navigation
├── load-testing         # bloque E — caos y carga
│   ├── fleet-simulation.js # script k6 (200 VUs, 5% inválidos, 10% duplicados)
│   └── results           # evidencia de corridas (summary JSON de k6)
├── infra               # bloque E — IaC de referencia (Terraform, no aplicado)
│   └── *.tf            # ECS Fargate, RDS, ALB, ECR
├── openspec            # disciplina Spec-Driven Development
│   ├── changes/archive   # 11 changes: proposal → specs → design → tasks
│   └── specs             # specs vigentes por capacidad, sincronizadas
├── info               # branding del cliente (logo, colores, mockup)
├── AGENTS.md          # contrato de reglas para el IDE agéntico
├── PLAN.md            # plan de ejecución fijado antes de codear
├── README.md          # arquitectura, cómo ejecutar, auditoría de IA
└── docker-compose.yml  # orquestación local de todos los servicios

```

La correspondencia 1:1 entre carpeta de primer nivel y bloque del enunciado (A→ `backend/ingestion-service` , B→ `backend/api-gateway` , C→ `frontend` , D→ `mobile` , E→ `load-testing` / `infra` ) no es casual: cada change de OpenSpec de la sección 7 delimitó su propio alcance de carpetas, lo que mantuvo el historial de commits legible por bloque incluso con IA agéntica escribiendo la mayoría del código.

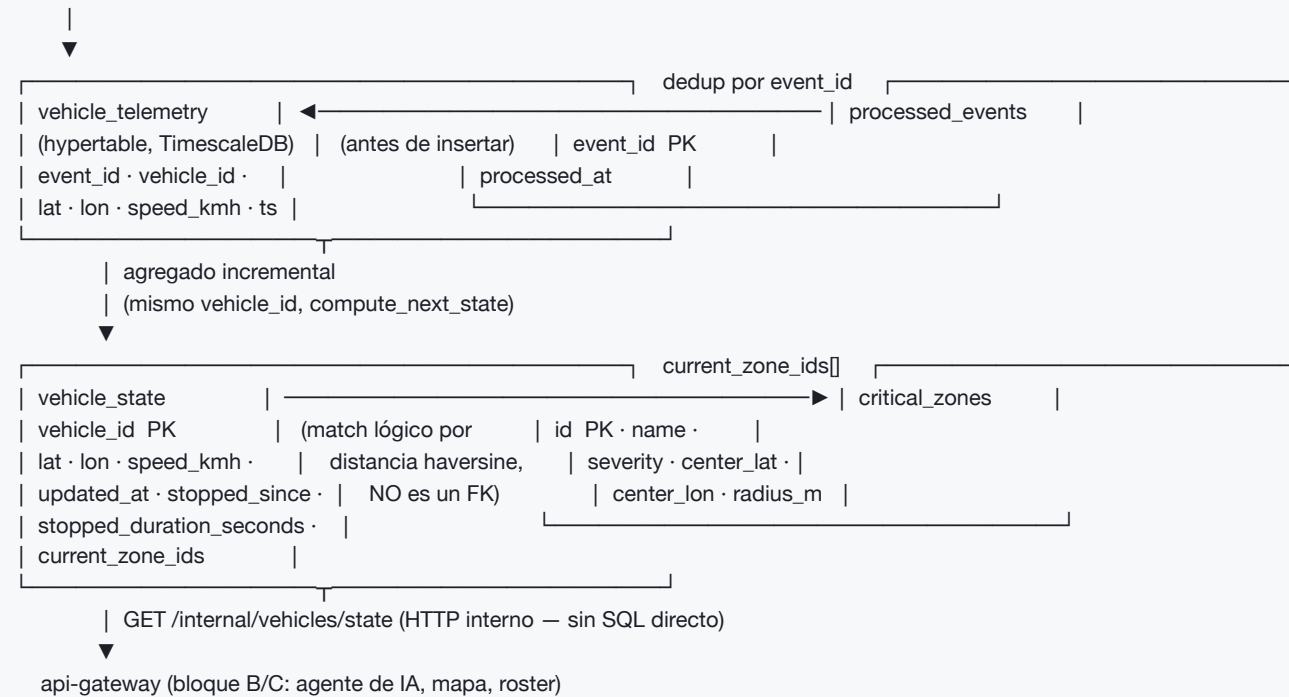
## 6.1 — Modelo de datos: tablas de persistencia y su relación

Toda la persistencia del sistema vive en una única base `TimescaleDB` (Postgres + extensión de series de tiempo), propiedad exclusiva de `ingestion-service` — `api-gateway` no tiene connection string propia ni

ejecuta SQL directo: consulta el estado de flota vía `GET /internal/vehicles/state` (HTTP interno), respetando el límite de bounded context entre los dos servicios del bloque A y B.

| Tabla             | Rol                                                   | Columnas clave                                                                                                    | Particularidad                                                                                                                                                               |
|-------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| vehicle_telemetry | Historial crudo de cada evento de telemetría recibido | event_id , vehicle_id , lat , lon , speed_kmh , ts                                                                | Hypertable particionado por ts , con retención de 7 días y compresión tras 1 día. Sin PRIMARY KEY compuesto — decisión corregida tras la Auditoría de IA (caso 2, sección 9) |
| processed_events  | Deduplicación de eventos por idempotency key          | event_id (PK), processed_at                                                                                       | Tabla separada del hypertable porque TimescaleDB exige que toda unique constraint incluya la columna de partición ( ts ), lo cual no sirve para deduplicar solo por event_id |
| vehicle_state     | Estado actual y agregado por vehículo                 | vehicle_id (PK), lat , lon , speed_kmh , updated_at , stopped_since , stopped_duration_seconds , current_zone_ids | Una sola fila por vehículo (upsert), recalculada incrementalmente por el consumer en cada evento nuevo — es la tabla que consulta el agente de IA del bloque B               |
| critical_zones    | Geocercas de zonas críticas                           | id (PK), name , severity , center_lat , center_lon , radius_m                                                     | Datos semilla fijos (3 zonas de ejemplo en Bogotá) — no hay endpoint CRUD para gestionarlasy desde el portal                                                                 |

Redpanda (tópico telemetry.raw)



Flujo: cada evento crudo pasa por dedup contra `processed_events`, se persiste en `vehicle_telemetry` y actualiza incrementalmente `vehicle_state` — que a su vez referencia `critical_zones` por coincidencia geográfica, no por clave foránea.

### SIN CLAVES FORÁNEAS EXPLÍCITAS

Ninguna de las cuatro tablas tiene una constraint `FOREIGN KEY` — la relación entre `vehicle_telemetry` / `vehicle_state` (mismo `vehicle_id`) y entre `vehicle_state` / `critical_zones` (`current_zone_ids` contra `critical_zones.id`) es lógica, resuelta en la capa `application` del consumer (`stop_detection.py`, `zone.py`), no por el motor de base de datos. Es una decisión deliberada, no un descuido: TimescaleDB restringe fuertemente las constraints que cruzan la columna de partición de un hypertable, y el dominio prioriza velocidad de ingesta de series de tiempo de alta frecuencia sobre integridad referencial estricta a nivel de esquema.

## 7. Ejecución por bloques: los 5 changes de OpenSpec

Cada bloque del enunciado se ejecutó como un change de OpenSpec independiente, en orden de dependencia (A es prerequisite de B, C y E; B y C dependen de A; D es paralelo a B/C):

### Bloque A

#### telemetry-ingestion

Servicio `ingestion-service` (FastAPI): `POST /v1/telemetry` valida y publica a Redpanda (tópico `telemetry.raw`); un consumer idempotente (dedup por `event_id`) persiste en TimescaleDB. Se modela explícitamente `CriticalZone` (geofence con datos semilla) y el estado derivado `stopped_since` / `stopped_duration` por vehículo — sin estos dos conceptos, el agente del bloque B no podría responder la pregunta de aceptación del PDF. Circuit breaker de base y skeleton de Docker Compose (Redpanda + TimescaleDB).

### Bloque B

#### fleet-ai-agent

Segundo servicio, `api-gateway`: endpoint de chat `POST /v1/agent/chat` (streaming SSE) con tool-use directo sobre `query_vehicle_state`, que consulta el estado interno de `ingestion-service` y filtra por zona crítica y tiempo detenido. Llamadas salientes (a `ingestion-service` y al proveedor de IA) envueltas en el Circuit Breaker compartido. API key exclusivamente server-side.

### Bloque C

#### web-portal-dashboard

SPA en `frontend/`: mapa (Leaflet) con posición de cada vehículo, panel de alertas derivado del mismo stream (sin nuevo concepto de "alerta" en backend), chat conectado al agente. Se agrega `GET /v1/vehicles/stream` (SSE) en `api-gateway` para evitar polling. Branding aplicado desde `./info/`: tema oscuro ( `#00ffc2` / `#19b5ff` ), logo, favicon y fondos.

### Bloque D

#### driver-mobile-app

App Expo/React Native: captura de GPS ( `expo-location` ), cola offline-first en SQLite — cada posición se guarda localmente primero y se envía si hay red, o queda pendiente si no la hay —, sincronización en bloque al reconectar ( `netinfo` detecta el cambio) contra un nuevo endpoint `POST /v1/telemetry/bulk`. CI/CD con GitHub Actions: validación (type-check) en cada push/PR, y un job de `eas build` documentado y gateado por secret, sin ejecutar un build real de tienda (no aplica a un MVP evaluado en 8-12h).

## Bloque E

### chaos-load-testing-iac

Script k6 ( `load-testing/fleet-simulation.js` ) simulando ~200 vehículos concurrentes contra `POST /v1/telemetry` , inyectando 5% de payloads inválidos y 10% de duplicados (mismo `event_id` ) para validar el dedup real del sistema. Terraform de referencia (ECS Fargate, RDS/Timescale, ALB, ECR), documental. Hardening final de `docker-compose.yml` y del `README.md` .

## 8. Iteraciones de refinamiento posteriores al MVP

Con los 5 bloques del enunciado funcionalmente completos y verificados, se ejecutaron 6 changes adicionales de OpenSpec — bajo la misma disciplina de `proposal` → `specs` → `design` → `tasks` → `archive` — para pasar de "MVP funcional" a "producto demostrable", motivados por pruebas reales en dispositivo físico y feedback directo del propio proceso de verificación:

| Change                                                                     | Motivación real                                                                                                                                                                                                                      | Qué resolvió                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>driver-experience-visual-refresh</code>                              | El portal y la app eran funcionalmente correctos pero visualmente crudos; el cliente entregó un mockup ( <code>info/simon_app_mockup_preview.html</code> ) con un lenguaje visual ya validado.                                       | Hero card de estado de flota, roster de vehículos con datos de presentación, "Guantera Digital", y en la app móvil: navegación por pestañas (Inicio/Rastreo/Guantera/SOS) en vez de un formulario único.                                              |
| <code>mobile-tracking-map</code>                                           | La pantalla de Rastreo mostraba un placeholder estático (grid CSS + círculo), no la posición GPS real.                                                                                                                               | Mapa real (Leaflet vía <code>react-native-webview</code> , compatible con Expo Go sin dev client) con la posición GPS real del dispositivo.                                                                                                           |
| <code>vehicle-live-telemetry-feed</code>                                   | Durante la verificación en dispositivo físico, la única forma de confirmar que los eventos GPS llegaban de verdad era consultar TimescaleDB por <code>psql</code> a mano.                                                            | Endpoint SSE <code>GET /v1/vehicles/{vehicle_id}/events/stream</code> + panel en el portal que muestra, seleccionando un vehículo, cada envío crudo de telemetría en tiempo real.                                                                     |
| <code>map-focus-on-feed-event</code>                                       | El feed de envíos en vivo era solo texto: no había forma de ver <i>dónde</i> ocurrió un envío puntual sin leer coordenadas mentalmente.                                                                                              | Click en una fila del feed centra y hace zoom el mapa a esa posición, con marcador destacado.                                                                                                                                                         |
| <code>mobile-web-dummy-profile-match / mobile-web-vehicle-id-config</code> | El perfil dummy de la app móvil y el del portal no coincidían para el mismo vehículo durante la demo en dispositivo real; luego, el <code>vehicle_id</code> hardcodeado quedaba desactualizado cada vez que cambiaba en el teléfono. | Coincidencia de perfil entre ambos clientes; y el <code>vehicle_id</code> "conocido" pasa de constante en código a variable de entorno ( <code>VITE_MOBILE_VEHICLE_ID</code> ) — cambio de configuración, no de código, cuando cambia en el teléfono. |

Estos 6 changes son evidencia directa de un patrón deliberado: cada problema encontrado **probando en vivo** (no leyendo código ni con tests unitarios) se convirtió en un nuevo change, no en un parche improvisado fuera del flujo de OpenSpec.

## 9. Auditoría de IA: criterio propio sobre código generado

Este es el entregable que el enunciado marca como **fundamental**: evidencia de que el código generado por el IDE agéntico fue auditado, no aceptado a ciegas. Se documentaron **7 casos reales** (el mínimo exigido era 2), cada uno con el problema, cómo se detectó y cómo se corrigió:

| # | Lo que sugirió la IA                                                                                           | Por qué era deficiente                                                                                                   | Cómo se detectó                                                                                                 |
|---|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| 1 | Columna <code>event_id</code> tipada como <code>UUID</code>                                                    | El dominio solo exige un string no vacío como idempotency key, no un formato UUID                                        | Ingesta en vivo con un id realista ( <code>evt-&lt;timestamp&gt;-1</code> ) falló con <code>invalid UUID</code> |
| 2 | <code>PRIMARY KEY (vehicle_id, ts)</code> en <code>vehicle_telemetry</code>                                    | Podía descartar silenciosamente dos eventos distintos con el mismo timestamp                                             | Comparando <code>COUNT(*)</code> vs <code>COUNT(DISTINCT event_id)</code> tras la prueba de carga k6            |
| 3 | <code>api-gateway</code> sin CORS habilitado                                                                   | <code>curl</code> no lo detecta — CORS es una política solo del navegador                                                | Probando el dashboard en un navegador real                                                                      |
| 4 | Manejo de errores incompleto en 2 endpoints tras integrar el Circuit Breaker                                   | Un <code>500</code> genérico en vez de <code>503</code> ; un fallo real de la API de IA rompía el SSE en vez de degradar | Apagando <code>ingestion-service</code> y probando con una cuenta sin créditos                                  |
| 5 | <i>(decisión confirmada, no revertida)</i> Circuit Breaker a mano vs. librería externa                         | —                                                                                                                        | Evaluated antes de escribir código; tenerlo propio permitió auditar y corregir el caso 4 con confianza          |
| 6 | <code>genai.Client</code> construido de forma anticipada en el arranque                                        | Valida la API key de forma síncrona y crashea el servicio si está vacía                                                  | Suite de <code>pytest</code> fallando en cualquier entorno sin <code>GEMINI_API_KEY</code>                      |
| 7 | Modelo <code>gemini-2.5-flash</code> hardcodeado; historial de conversación sin <code>thought_signature</code> | <code>404</code> en API keys nuevas; <code>400</code> exigiendo un campo opaco no documentado obviamente                 | Probando con una API key real end-to-end, no con tests unitarios                                                |

PATRÓN QUE SE REPITE EN 5 DE LOS 7 CASOS

El error no se encontró leyendo el código generado ni con tests unitarios aislados — se encontró **ejecutando el sistema completo** (prueba de carga real, dispositivo físico real, API real, navegador real). Es la evidencia más concreta de que la IA agéntica acelera la escritura de código sintácticamente correcto, pero el criterio senior sigue siendo insustituible para detectar lo que solo falla en producción.

10. Pruebas y verificación funcional

| Verificación                                                      | Resultado                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Carga y caos (k6, <code>load-testing/fleet-simulation.js</code> ) | ~200 VUs concurrentes, ~29 mil iteraciones, <b>100% de checks exitosos</b> . 10% de duplicados y 5% de payloads inválidos inyectados según el enunciado. <code>COUNT(*) = COUNT(DISTINCT event_id)</code> confirmado tras la corrida: cero duplicados reales persistidos, cero pérdida de datos. |
| Agente de IA en vivo                                              | Verificado con <code>GEMINI_API_KEY</code> real preguntando literalmente la pregunta de aceptación del PDF ("¿qué vehículos llevan detenidos más de 20 minutos en zonas críticas?") — responde correctamente identificando vehículo, zona y tiempo detenido.                                     |
| App móvil en dispositivo físico (iPhone, Expo Go)                 | Captura de GPS real, cola offline y sincronización en bloque confirmadas de punta a punta: el dispositivo capturó posiciones estando desconectado y, al reconectar, disparó un único <code>POST /v1/telemetry/bulk</code> deduplicado y persistido correctamente.                                |
| Tests automatizados                                               | Suites de <code>pytest</code> en <code>ingestion-service</code> y <code>api-gateway</code> (incluye el Circuit Breaker), type-check estricto ( <code>tsc</code> ) en frontend y mobile.                                                                                                          |
| Infraestructura (Terraform)                                       | <code>terraform validate</code> exitoso sin credenciales de AWS — validación sintáctica del despliegue de referencia, sin aplicarlo.                                                                                                                                                             |

11. Estado de entregables

La sección 7 del enunciado exige cuatro entregables concretos. Los cuatro están completos:

| # | Entregable                                            | Estado   | Evidencia                                                                                                       |
|---|-------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------|
| 1 | Repositorio Git con historial de commits estructurado | COMPLETO | 137 commits en <code>main</code> , Conventional Commits, sincronizado con <code>origin</code> (GitHub, público) |

| # | Entregable                                           | Estado   | Evidencia                                                                                                                                   |
|---|------------------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | README.md con instrucciones de ejecución e IaC       | COMPLETO | Arquitectura, setup vía Docker Compose, prueba end-to-end manual, Terraform                                                                 |
| 3 | Auditoría de IA (mínimo 2 casos)                     | COMPLETO | 7 casos documentados en README.md, con referencia a commits <code>fix(...)</code>                                                           |
| 4 | Video de sustentación (5-10 min, YouTube no listado) | COMPLETO | Arquitectura, demostración funcional y entorno de trabajo agéntico ( AGENTS.md , flujo <code>openspec new change → apply → archive</code> ) |

## 12. Conclusión

El proyecto demuestra el objetivo que el propio enunciado fija como criterio de evaluación: usar IA agéntica como *exo esqueleto*, no como muleta. La disciplina de Spec-Driven Development con OpenSpec convirtió cada bloque del enunciado en un ciclo verificable de propuesta → especificación → diseño → implementación → archivo, y dejó un historial de git que es, en sí mismo, una bitácora auditable de decisiones — no solo de código.

Los 7 casos de la Auditoría de IA muestran un patrón consistente: el código generado por la IA era sintácticamente correcto pero fallaba bajo condiciones reales (carga real, dispositivo real, API real, navegador real) que solo aparecen al ejecutar el sistema completo, no leyendo el diff. Ese es, precisamente, el criterio senior que el enunciado pide demostrar.